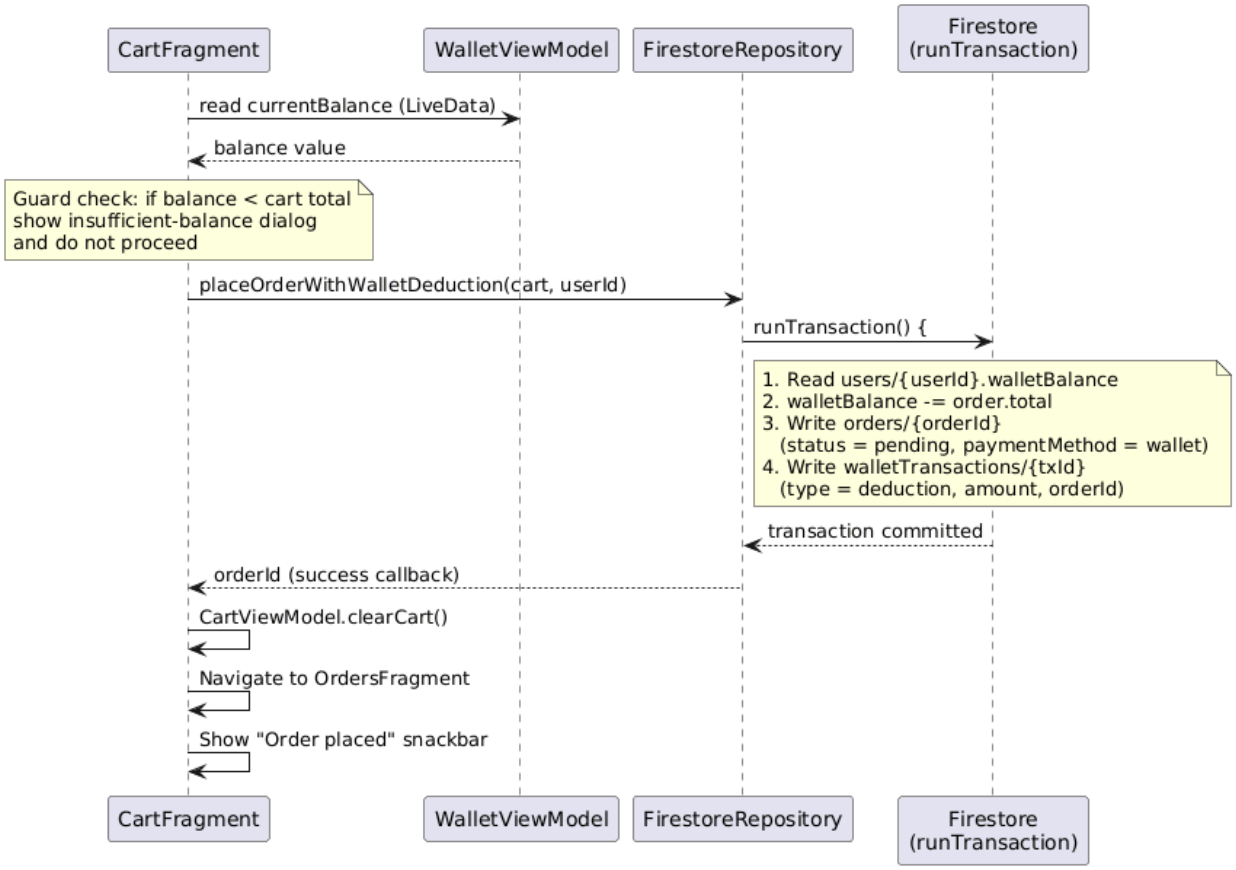

Figure 35: Cart-Wallet Integration — placeOrderWithWalletDeduction()

Figure 35 shows the cart-to-order transaction sequence. The runTransaction encompasses four operations - reading the current balance, writing the deducted balance, writing the order document, and writing the walletTransactions record - as a single atomic unit on the Firestore server. This guarantees that no partial state can persist regardless of network conditions or concurrent requests from other devices.

iii. Order Status Flow Integration

The order status lifecycle integrates the staff's Android interface, Firestore, the Supabase notify-order-ready Edge Function, and the student's Android interface through a coordinated sequence of events. When a staff member taps "Mark Ready" in StaffOrdersFragment, the fragment calls FirestoreRepository.updateOrderStatus(orderId, "ready"), which writes the new status to the Firestore orders document. This write triggers an immediate update in the student's OrdersFragment through the active listenToStudentOrders() snapshot listener, changing the status

chip from blue ("Preparing") to green ("Ready"). Concurrently, after the Firestore write completes successfully, the Android app calls FirestoreRepository.notifyOrderReady(orderId, userId) as a background operation. This method sends an HTTP POST to the notify-order-ready Supabase Edge Function with the FUNCTIONS_SECRET Bearer token. The edge function reads the student's fcmToken from Firestore, constructs an FCM notification payload, and dispatches the push notification to the student's device.

This integration means the student receives both a real-time in-app UI update (via the Firestore listener) and a push notification (via FCM), regardless of whether the app is foregrounded. The notification call to the edge function is fail-silent: if the network request fails, the order status has already been updated in Firestore, so the student's in-app view will still reflect the "Ready" state. The notification is a supplementary alert, not the primary status mechanism.

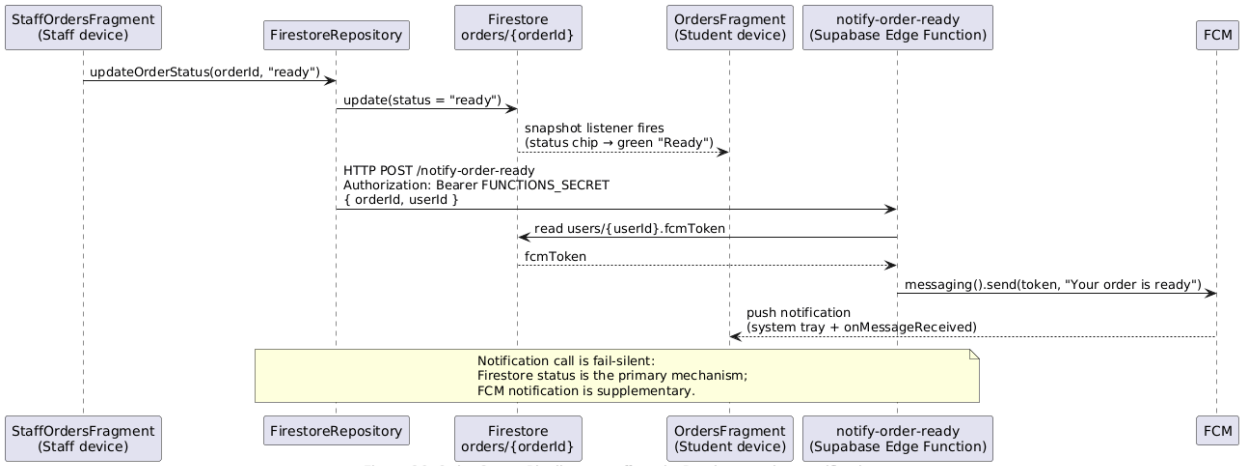

Figure 36: Order Status Pipeline — staff marks Ready to student notification

Figure 36 shows the order status pipeline. The Firestore write is the primary mechanism driving the student's in-app update; the subsequent HTTP POST to the edge function is a supplementary notification path. Because the student's OrdersFragment holds an active snapshot listener, the status chip transitions from blue to green within milliseconds of the staff action, independent of FCM delivery.

iv. Pre-order Cut-off Integration

The pre-order cut-off workflow integrates cron-job.org, the Supabase process-cutoff Edge Function, Firestore, and Firebase Cloud Messaging into a fully automated pipeline that runs without any user or administrator intervention.

Two cron jobs are configured in cron-job.org. The first executes at 07:00 UTC daily (10:00 AM East Africa Time) and sends an HTTP POST to the process-cutoff Edge Function with a JSON body specifying mealSlot as "lunch". The second executes at 14:00 UTC daily (5:00 PM East Africa Time) with mealSlot set to "dinner". Each request includes the FUNCTIONS_SECRET Bearer token for authentication.

Upon receiving the request, the process-cutoff function queries Firestore for all preOrders documents where status is "scheduled", scheduledDate equals today's date, and mealSlot matches the request parameter. For each matching document, the function performs the following steps inside a Firestore transaction: it reads the student's current walletBalance; if the balance is greater than or equal to the pre-order's totalAmount, it deducts the amount, sets the pre-order status to "confirmed", and creates a walletTransactions document of type "deduction"; if the balance is insufficient, it sets the status to "cancelled" without touching the wallet. In both cases, an FCM notification is sent to the student's device via the fcmToken stored in their users document - a confirmation message if confirmed, a cancellation alert with a request to top up if cancelled.

For recurring pre-orders (recurring == true), after processing the current document, the function creates a new preOrders document with the same items, mealSlot, and recurringDays but with scheduledDate set to the same day of the following week and status set to "scheduled". This new document will be picked up automatically at the next week's cut-off run, creating a continuous weekly chain that persists until the student cancels a scheduled instance.

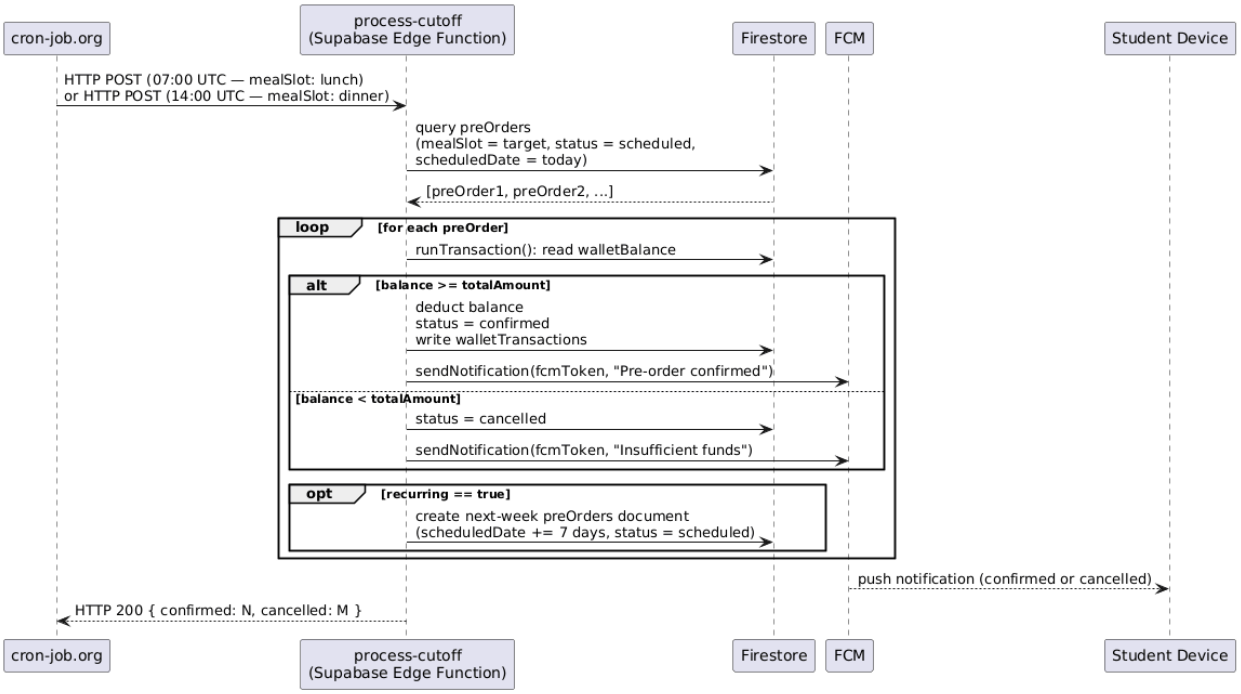

Figure 37: Pre-order Cut-off Integration — end-to-end flow

Figure 37 illustrates the end-to-end cut-off flow. The student has no involvement in this process beyond the initial scheduling; the wallet deduction, status update, and notification all occur automatically at the correct time each day.

6.5 APIs

The USIU Cafeteria Ordering System integrates a set of Firebase APIs, the Supabase Edge Functions HTTP interface, and the Firebase Admin SDK to enable authentication, real-time data synchronisation, push notifications, and server-side account management. Each API plays a distinct role in the system's operation, and together they form the complete communication layer between the two client applications and the cloud backend.

1. Firebase Authentication API

The Firebase Authentication API handles user identity across both client applications. In the Android app, LoginActivity uses the signInWithEmailAndPassword method from the FirebaseAuth class to authenticate students and staff. RegisterActivity uses createUserWithEmailAndPassword to create new student accounts, followed by a Firestore write to create the accompanying users document. ChangePasswordActivity uses the updatePassword

method on the currently authenticated FirebaseUser object to enforce the mandatory first-login password change for staff accounts.

In the Next.js admin panel, the Firebase Client SDK's signInWithEmailAndPassword is used on the login page. The admin panel's API routes use the Firebase Admin SDK's auth().createUser() method to provision new staff accounts server-side, avoiding the need to expose administrative credentials in the client browser.

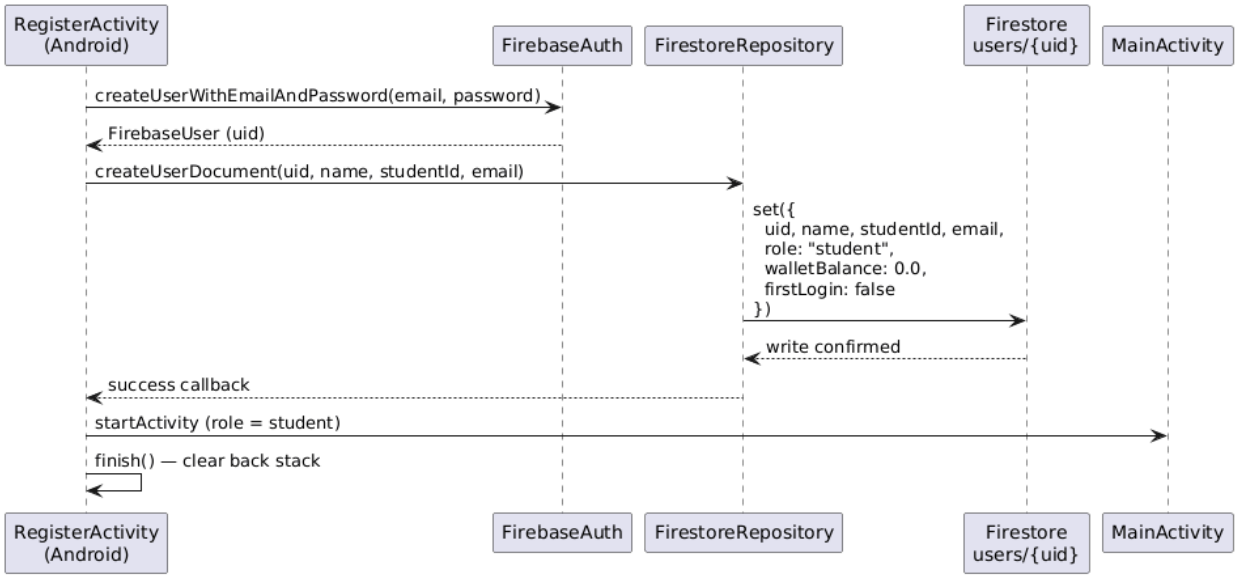

Figure 38: Firebase Authentication API — Student Registration Flow

Figure 38 illustrates the student registration sequence. After Firebase Auth creates the user, the app calls FirestoreRepository to write the users document synchronously before redirecting to MainActivity. This ensures the role field is available immediately on the user's first login.

2. Firebase Firestore SDK

The Firebase Firestore SDK is the most extensively used API in the system, underpinning all data reads, writes, and real-time synchronisation in the Android app. The key Firestore SDK methods used are:

addSnapshotListener attaches a persistent, real-time listener to a collection or query result. The listener fires immediately with the current data and then again whenever the underlying documents

change. This is the mechanism behind the live order status updates in the student's OrdersFragment and the live order queue in StaffOrdersFragment.

runTransaction executes a server-side atomic operation. All wallet balance modifications - top-ups, order deductions, staff-initiated debits, and pre-order deductions at cut-off - use runTransaction to guarantee that no concurrent operation can produce an inconsistent balance. The transaction reads the current balance, computes the new value, and writes it back in a single atomic step.

set and update write documents and individual fields to Firestore. The repository uses set for new documents (orders, preOrders, walletTransactions) and update for field-level changes such as order status transitions and availability toggles on menu items.

Composite indexes deployed to Firestore enable the compound queries required by the system, such as querying orders by userId and sorting by createdAt in a single operation, or querying preOrders by mealSlot, scheduledDate, and status simultaneously for the cut-off function.

```java
/** Real-time listener for a student's own orders, newest first. */
public ListenerRegistration listenToMyOrders(String userId,
                                             EventListener<QuerySnapshot> listener) {
    return db.collection("orders")
            .whereEqualTo("userId", userId)
            .orderBy("createdAt", com.google.firebase.firestore.Query.Direction.DESCENDING)
            .addSnapshotListener(listener);
}

/** Real-time listener for all active orders (staff view). */
public ListenerRegistration listenToAllActiveOrders(EventListener<QuerySnapshot> listener) {
    return db.collection("orders")
            .whereIn("status", Arrays.asList("pending", "preparing", "ready"))
            .orderBy("createdAt", com.google.firebase.firestore.Query.Direction.ASCENDING)
            .addSnapshotListener(listener);
}

public Task<Void> updateOrderStatus(String orderId, String newStatus, String userId) {
    Task<Void> task = db.collection("orders")
            .document(orderId)
            .update("status", newStatus);

    if ("ready".equals(newStatus)) {
        task.addOnSuccessListener(unused -> notifyOrderReady(userId));
    }

    return task;
}
```

Figure 39 shows the addSnapshotListener call within listenToStudentOrders(). The listener is attached once when the ViewModel initialises and remains active for the lifetime of the student's session. Each QuerySnapshot delivered by Firestore is mapped to a list of Order objects using toObject(Order.class) and posted to the MutableLiveData, which OrdersFragment observes to update its RecyclerView without any manual refresh.

3. Firebase Cloud Messaging (FCM) SDK

The Firebase Cloud Messaging SDK handles push notification delivery from the Supabase Edge Functions to student devices. On the Android client side, a FirebaseMessagingService subclass overrides the onNewToken callback to capture the device's FCM registration token whenever it is refreshed. The new token is immediately written to the fcmToken field of the authenticated user's

Firestore document, ensuring the edge functions always have access to an up-to-date delivery address.

Push notifications are sent from the Supabase Edge Functions using the Firebase Admin SDK's messaging().send() method, which accepts a notification payload and a target FCM token. Notifications are received by the Android OS and displayed as system notifications even when the app is in the background. When the app is in the foreground, the onMessageReceived callback allows the app to handle the notification programmatically - for example, by refreshing the Orders tab to immediately show the updated status.

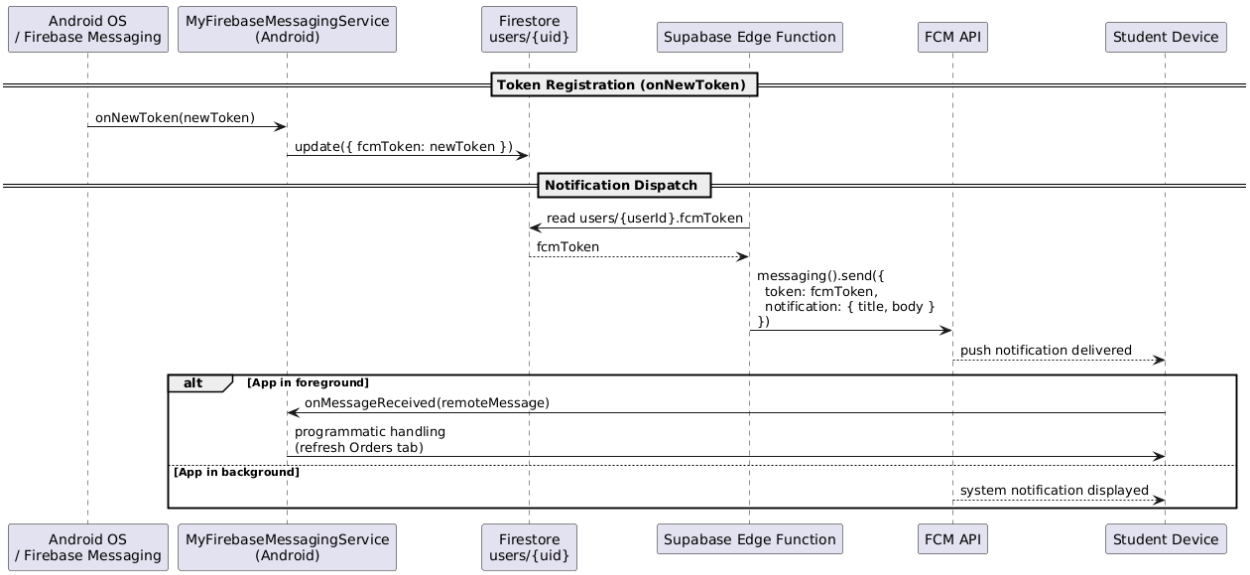

Figure 40: FCM SDK Integration — token registration and notification delivery

Figure 40 illustrates the two-phase FCM integration. The onNewToken callback runs automatically whenever Firebase rotates the device's registration token; the immediate Firestore write ensures edge functions always hold a valid delivery address. The notification dispatch path is entirely server-side, keeping the student device's role passive - it only receives notifications, never initiates them.

4. Supabase Edge Functions HTTP API

The Supabase Edge Functions are exposed as standard HTTPS endpoints and are consumed via plain HTTP POST requests. There is no client SDK - both the Android app and cron-job.org interact with the functions using HTTP directly.

The Android app's FirestoreRepository.notifyOrderReady() method constructs an HTTP POST request using Java's HttpURLConnection or OkHttp, setting the Content-Type header to application/json and the Authorization header to Bearer followed by the FUNCTIONS_SECRET value retrieved from BuildConfig. The request body is a JSON object containing the orderId and userId. The call is made on a background thread to avoid blocking the main UI thread.

The cron-job.org service sends HTTP POST requests to the process-cutoff endpoint with a JSON body specifying the mealSlot. These requests are authenticated with the same FUNCTIONS_SECRET Bearer token. The edge functions respond with an HTTP 200 status and a JSON summary of the processing result (e.g, number of confirmed and cancelled pre-orders), which cron-job.org logs for monitoring purposes.

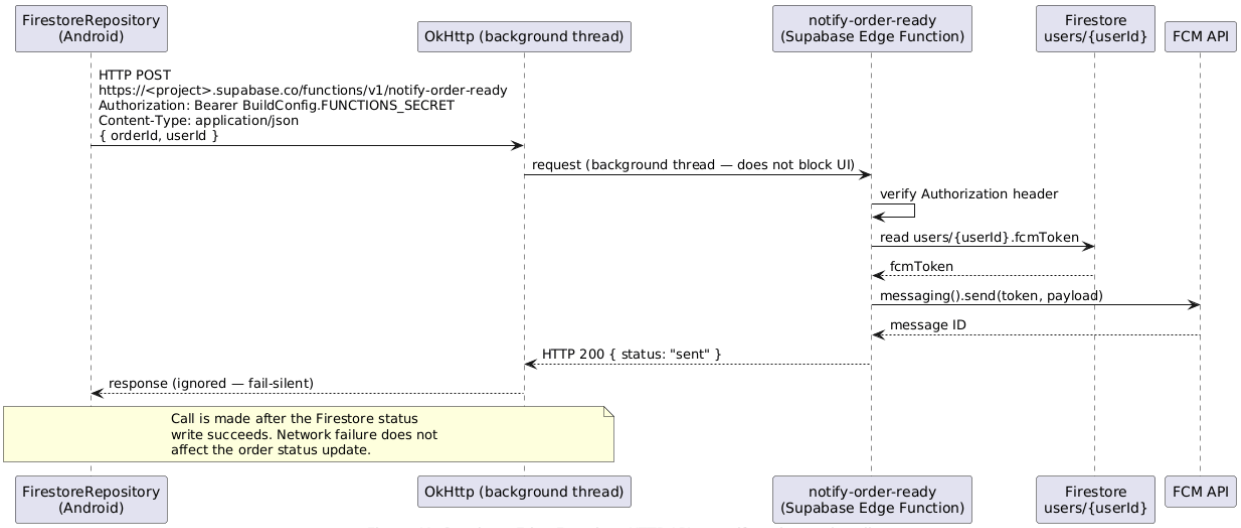

Figure 41: Supabase Edge Functions HTTP API — notify-order-ready call

Figure 41 shows the HTTP API call from the Android app to the notify-order-ready edge function. The call is dispatched on a background thread immediately after the Firestore status write, using the FUNCTIONS_SECRET stored in BuildConfig to authenticate the request. The response is ignored by the app; if the network call fails, the student's in-app status chip already reflects the updated state through the active Firestore snapshot listener.

5. Firebase Admin SDK (Next.js API Routes)

The Firebase Admin SDK is used exclusively in the Next.js admin panel's server-side API routes. Unlike the client-side Firebase SDK, the Admin SDK bypasses Firestore security rules and Firebase Authentication role restrictions, allowing it to create Auth users with any role and read or write any Firestore document. This elevated privilege is why the Admin SDK is restricted to server-side code only; its service account credentials are stored as environment variables in Vercel and are never exposed to the browser.

The create-staff API route initialises the Admin SDK with the service account credentials, calls auth().createUser() to provision the Firebase Authentication account with the specified email and a temporary password, then calls firestore().collection("users").doc(newUid).set() to create the Firestore users document with role: "staff" and firstLogin: true. The route returns the new user's UID to the admin panel client on success, or a structured error message on failure.

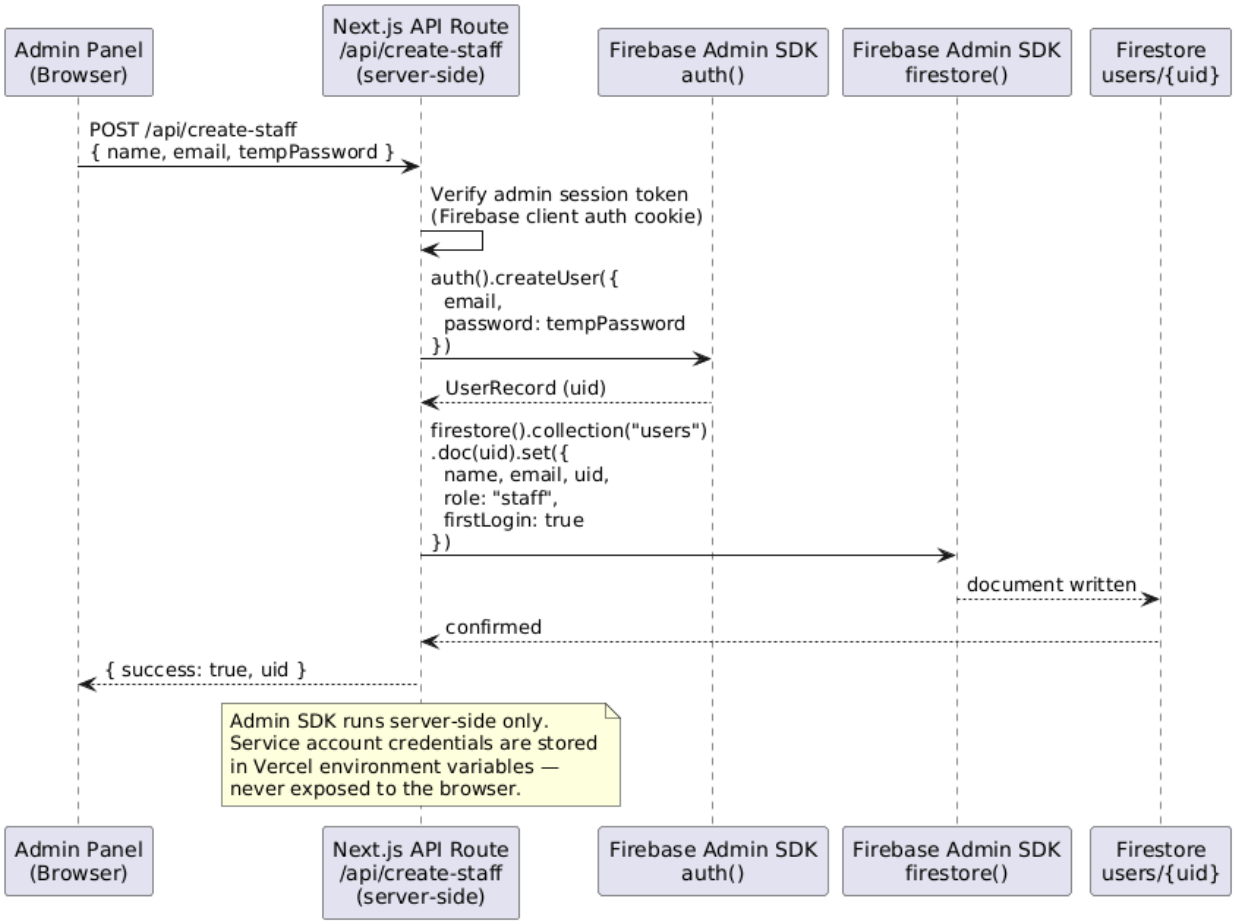

Figure 42: Firebase Admin SDK — Staff Account Creation via Next.js API Route

Figure 42 illustrates the staff account creation API route. The server-side execution context ensures that the Admin SDK credentials are never accessible to browser clients, maintaining the security boundary between the admin panel's user interface and its privileged backend operations.

6.6 Application Deployment

The deployment phase of the USIU Cafeteria Ordering System packaged and published all components of the system into their respective production environments: the Android APK onto physical student and staff devices, the Supabase Edge Functions to the Supabase cloud platform, the scheduling jobs to cron-job.org, the Firestore security rules and indexes to the Firebase project, and the Next.js admin panel to Vercel. A key objective throughout this phase was to achieve full production deployment without incurring any infrastructure costs, utilising the free tiers of all platforms involved.

1. Gradle Build Configuration and Android APK Generation

The Android application's build configuration is defined in the module-level build.gradle file. The minSdkVersion is set to API Level 24 (Android 7.0), ensuring compatibility with the vast majority of student and staff devices on campus. The targetSdkVersion is set to the latest stable API level to leverage current Android security and performance features. The applicationId is set to com.usiu.cafeteria and the app is versioned for production release.

Dependencies declared in build.gradle include the Firebase BOM (Bill of Materials) for consistent Firebase SDK versioning, with individual Firebase modules for Firestore, Authentication, and Cloud Messaging. Additional dependencies include the Material Design 3 library, Glide for image loading with its annotation processor, and the AndroidX ViewModel and LiveData libraries for MVVM support.

The FUNCTIONS_SECRET value used to authenticate calls to the Supabase Edge Functions is stored in the module-level gradle.properties file, which is excluded from version control via .gitignore. It is exposed to the app at build time as a BuildConfig field, accessed in code as BuildConfig.FUNCTIONS_SECRET. This approach keeps the secret out of source control while making it available at runtime.

A signed release APK was generated using Android Studio's "Generate Signed Bundle/APK" wizard. A release keystore was created and used to sign the APK, a mandatory step for distribution and future update compatibility. The signed APK was then installed directly onto physical devices for testing and deployment.

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.google.services)
}

android {
    namespace 'com.usiu.cafeteria'
    compileSdk {
        version = release(36) {
            minorApiLevel = 1
        }
    }

    defaultConfig {
        applicationId "com.usiu.cafeteria"
        minSdk 26
        targetSdk 36
        versionCode 1
        versionName "1.0"

        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"

        // Supabase Edge Function config — fill in after deploying edge functions
        buildConfigField "String", "SUPABASE_NOTIFY_URL", "\"${project.findProperty('SUPABASE_NOTIFY_URL') ?: ''}\""
        buildConfigField "String", "FUNCTIONS_SECRET",    "\"${project.findProperty('FUNCTIONS_SECRET')    ?: ''}\""
    }

    buildFeatures {
        buildConfig true
    }

    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'
        }
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_11
        targetCompatibility JavaVersion.VERSION_11
    }
}

dependencies {
    implementation libs.appcompat
    implementation libs.material
    implementation libs.activity
    implementation libs.constraintlayout

    // Firebase (versions managed by BOM)
    implementation platform(libs.firebase.bom)
    implementation libs.firebase.firestore
    implementation libs.firebase.auth
    implementation libs.firebase.messaging

    // Image loading
    implementation libs.glide

    // ViewModel + LiveData
    implementation libs.lifecycle.viewmodel
    implementation libs.lifecycle.livedata

    testImplementation libs.junit
    androidTestImplementation libs.ext.junit
```

Figure 43 shows the module-level build.gradle configuration. The Firebase BOM ensures all Firebase SDK modules use compatible versions without requiring individual version declarations. The buildConfigField entry for FUNCTIONS_SECRET generates a compile-time constant from gradle.properties, keeping the secret out of source code while making it available at runtime as BuildConfig.FUNCTIONS_SECRET.

2. Firestore Security Rules Deployment

Firestore security rules enforce role-based access control at the database level, ensuring that students can only read their own data, staff can access order and menu data, and admin operations are blocked at the client SDK level (forcing them through the Admin SDK in Next.js API routes).

The rules define four helper functions: isStudent() checks that the authenticated user's role field in Firestore equals "student"; isStaff() checks for "staff"; isAdmin() checks for "admin"; and isPrivileged() returns true if the user is either staff or admin. These helpers are used throughout the rules to gate collection access. For example, the walletTransactions collection allows reads only to the owning student or privileged users, and write operations that modify walletBalance are blocked for any client with the student role, preventing self-top-up.

The rules were deployed to the Firebase project using the Firebase CLI's firebase deploy --only firestore:rules command, making them active immediately across all client connections.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function isAuth() { return request.auth != null; }
    function role() {
      return get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role;
    }
    function isStaff()  { return isAuth() && role() == 'staff';  }
    function isAdmin()  { return isAuth() && role() == 'admin';  }
    function isPrivileged() { return isStaff() || isAdmin(); }
    function isOwner(uid) { return isAuth() && request.auth.uid == uid; }

    match /users/{userId} {
      allow read: if isOwner(userId) || isPrivileged();
      allow create: if isAuth() && isOwner(userId);
      // walletBalance changed by staff; role changed by admin
      allow update: if isPrivileged();
    }

    match /menuItems/{itemId} {
      allow read: if isAuth();
      allow write: if isPrivileged();
    }

    match /orders/{orderId} {
      allow read: if isPrivileged() || (isAuth() && resource.data.userId == request.auth.uid);
      allow create: if isAuth() && request.resource.data.userId == request.auth.uid;
      allow update: if isPrivileged();
    }

    match /preOrders/{preOrderId} {
      allow read: if isPrivileged() || (isAuth() && resource.data.userId == request.auth.uid);
      allow create: if isAuth() && request.resource.data.userId == request.auth.uid;
      allow update: if isPrivileged() ||
        (isAuth() && resource.data.userId == request.auth.uid
          && request.resource.data.status == 'cancelled');
    }

    match /walletTransactions/{txId} {
      allow read: if isOwner(resource.data.userId) || isPrivileged();
      allow write: if isPrivileged();
    }
  }
}
```

Figure 44 shows the deployed Firestore security rules. The four helper functions consolidate role checks into reusable expressions, keeping individual collection rules concise. The isPrivileged() helper in particular allows staff and admin access to be granted with a single function call rather than duplicating role comparisons across every collection block, reducing the risk of misconfiguration.

3. Firestore Composite Indexes Deployment

Five composite indexes were created in the Firebase Console's Firestore Indexes section to support the compound queries used by the application. Firestore requires explicit index definitions for any query that combines a field filter with an ordering on a different field, or that applies multiple equality or range filters simultaneously.

The deployed indexes are: userId ASC + createdAt DESC on the orders collection (for student order history); status ASC + createdAt ASC on orders (for the staff order queue); userId ASC + createdAt DESC on preOrders (for student pre-order listing); mealSlot ASC + scheduledDate ASC + status ASC on preOrders (for the cut-off function query); and userId ASC + createdAt DESC on walletTransactions (for wallet transaction history). Without these indexes, the corresponding Firestore queries would fail at runtime with a "requires an index" error.

4. Supabase Edge Functions Deployment

The two Supabase Edge Functions - process-cutoff and notify-order-ready - were deployed to the Supabase project using the Supabase CLI (supabase functions deploy). Each function resides in its own directory under supabase/functions/ and is deployed independently. The CLI packages the TypeScript source, uploads it to the Supabase platform, and makes it immediately available at its assigned HTTPS endpoint.

The FUNCTIONS_SECRET and the Firebase service account credentials (a JSON key file) are stored in the Supabase project as secrets using the supabase secrets set command. The secrets are injected into the edge function's Deno runtime as environment variables, accessible via Deno.env.get(). This keeps all sensitive credentials off the codebase and out of version control entirely.

```
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin\Code\beni-2>cd C:\Users\Admin\Code\beni-2

C:\Users\Admin\Code\beni-2>C:\supabase-cli\supabase.exe functions deploy process-cutoff --project-ref mjkowkfdneyegqtodxur
WARNING: Docker is not running
Uploading asset (process-cutoff): supabase/functions/process-cutoff/index.ts
Deployed Functions on project mjkowkfdneyegqtodxur: process-cutoff
You can inspect your deployment in the Dashboard: https://supabase.com/dashboard/project/mjkowkfdneyegqtodxur/functions

C:\Users\Admin\Code\beni-2>C:\supabase-cli\supabase.exe functions deploy notify-order-ready --project-ref mjkowkfdneyegqtodxur
WARNING: Docker is not running
Uploading asset (notify-order-ready): supabase/functions/notify-order-ready/index.ts
Deployed Functions on project mjkowkfdneyegqtodxur: notify-order-ready
You can inspect your deployment in the Dashboard: https://supabase.com/dashboard/project/mjkowkfdneyegqtodxur/functions

C:\Users\Admin\Code\beni-2>
```

Figure 45 shows the Supabase CLI deployment commands. Each supabase functions deploy call uploads the compiled TypeScript function and makes it immediately available at its HTTPS endpoint. The supabase secrets set commands inject the FUNCTIONS_SECRET and Firebase service account credentials into the Deno runtime as environment variables, keeping all sensitive values entirely off the codebase and out of version control.

5. cron-job.org Scheduling Configuration

Two cron jobs were created in cron-job.org to trigger the process-cutoff Edge Function on the required schedule. The first job is configured to run at 07:00 UTC daily, corresponding to 10:00 AM East Africa Time, and sends an HTTP POST to the process-cutoff URL with the request body containing the mealSlot field set to "lunch". The second job runs at 14:00 UTC daily (5:00 PM EAT) with mealSlot set to "dinner". Both jobs include the Authorization header with the Bearer token set to the FUNCTIONS_SECRET value.

cron-job.org provides a web dashboard showing the execution history, response code, and response body for each job run. This serves as the primary monitoring tool for the pre-order cut-off automation, allowing the administrator to verify that each day's processing completed successfully and to inspect the summary of confirmed and cancelled pre-orders returned by the edge function.

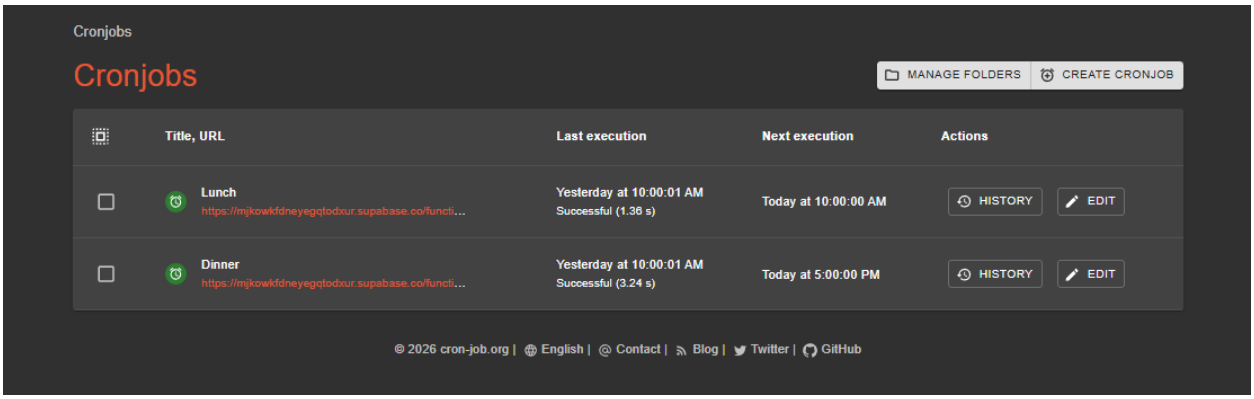

Figure 46 shows the cron-job.org scheduling configuration. The two jobs are staggered at 07:00 UTC and 14:00 UTC - corresponding to the 10:00 AM and 5:00 PM East Africa Time cut-offs - ensuring pre-orders are processed precisely at the configured deadlines. The execution history log provides a daily audit trail confirming that each cut-off run completed with an HTTP 200 response and a valid processing summary.

6. Next.js Admin Panel Deployment (Vercel)

The Next.js admin panel was deployed to Vercel by connecting the GitHub repository to a Vercel project. Vercel detects the Next.js framework automatically and configures the build pipeline accordingly. The admin panel is deployed from the cafeteria-app/admin/ subdirectory of the repository using Vercel's root directory configuration setting.

The Firebase service account credentials required by the Admin SDK in the Next.js API routes are stored as Vercel environment variables. These variables are injected into the Next.js server runtime at build and request time, and are never exposed to the browser. The client-side Firebase configuration (API key, project ID, etc.) is stored in public environment variables (prefixed NEXT_PUBLIC_) and is safe to include in the client bundle, as it grants access only within the bounds of the Firestore security rules.

The Vercel deployment is triggered automatically on each push to the main branch, making it straightforward to roll out updates to the admin panel. The admin panel is served over HTTPS by

default on Vercel's infrastructure, ensuring encrypted communication between the administrator's browser and the Next.js server.

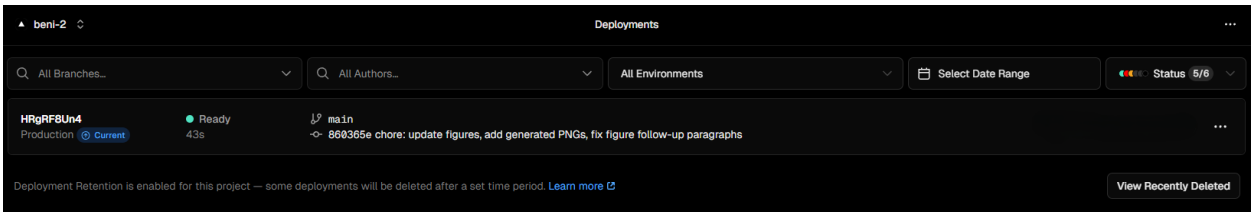

Figure 47 shows the Vercel deployment dashboard. Automatic deployment on each push to the main branch eliminates manual steps for admin panel updates. The environment variable list confirms that the Firebase service account credentials are stored securely server-side, while the NEXT_PUBLIC_ Firebase config values are separately scoped for safe client-side inclusion within the bounds of the Firestore security rules.

6.7 Conclusion

This chapter presented the complete implementation of the USIU Cafeteria Ordering System, documenting the construction, integration, and deployment of all components across the two client applications and the cloud backend. The Android application was built in Java using XML layouts and Material Design 3 components, structured around a single-activity MVVM architecture with a centralised FirestoreRepository handling all data access. The web-based admin panel was built with Next.js 14 and TypeScript, using the Firebase Admin SDK in server-side API routes to securely manage staff accounts and system data. Both applications share the same Firebase project and Firestore database, governed by a unified set of security rules that enforce role-based access for students, staff, and administrators.

The backend relies on Firebase Firestore for real-time data synchronisation, Firebase Authentication for identity management across all three roles, and Firebase Cloud Messaging for push notification delivery. Supabase Edge Functions written in TypeScript replace the Firebase Cloud Functions that the original design specified, enabling the scheduled pre-order cut-off automation and push notification dispatch to operate on the free Spark plan without requiring a

Firebase Blaze subscription. The critical technical insight that Deno's gRPC incompatibility requires the Firebase Admin SDK to be configured with preferRest: true was discovered and resolved during implementation, and is documented in the code and in the deployment notes.

All wallet operations - top-ups by staff, deductions for regular orders, and pre-order charges at cut-off - execute inside Firestore runTransaction calls, providing atomic, consistent, and isolated updates to wallet balances regardless of concurrent access from multiple devices. The hire/show fragment navigation pattern preserves UI state across tab switches, and the FirestoreRepository's snapshot listeners ensure that order status changes propagate to all affected screens in real time. Deployment was achieved entirely on free-tier infrastructure: Firebase Spark, Supabase free, cron-job.org free, and Vercel free, resulting in a production-ready system with zero recurring operational costs.

7 CHAPTER 7: TESTING AND EVALUATION

7.1 Introduction

This chapter presents the comprehensive testing and evaluation phase of the USIU Cafeteria Ordering System, encompassing unit-level logic verification, system testing, security rule validation, performance benchmarking, and user acceptance testing. The primary objective was to confirm that all features - from student registration and cart management through to real-time order tracking and FCM push notifications - behaved correctly and reliably on a physical Android device connected to the live Firebase backend.

Three categories of system testing were applied. Functional testing evaluated all nine core user scenarios, covering both the student and staff interfaces. Security testing validated the Firestore security rules using the Firebase Console Rules Playground to confirm that role-based access boundaries were correctly enforced. Performance testing measured key response times - including application launch, order placement propagation, and real-time status updates - to verify that the system met the responsiveness requirements defined during the design phase.

In addition, a UAT session was conducted in two stages: a developer-led role simulation using two devices simultaneously, and a peer review involving USIU-Africa students who used the application as real users. Feedback from these sessions informed the Key Findings and Impact Analysis sections that follow. Together, these testing activities confirmed that the system is functionally complete, secure, and ready for deployment as a working prototype.

7.2.1 Functional Testing

Functional testing verified that all nine core user scenarios performed correctly end-to-end on a physical Android device against the live Firebase Firestore and FCM backend. The test cases covered the student order lifecycle, cash and wallet payment paths, staff order management, pre-order scheduling, and wallet top-up by staff.

Test Cases - Table 7.1: Functional Testing

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC01	Student registration	Open app, tap Register, enter name, student ID (e.g 671234), email and password, tap Register	Lands on Menu screen; user document created in Firestore with role "student" and walletBalance 0	Registered successfully; Menu screen loaded with all items; Firestore document confirmed in console	**Passed**
TC02	Cart management	Add "Beef Stew" and "Chapati" to cart, increase Chapati quantity to 2, remove Beef Stew, verify total	Cart shows Chapati x2 only; total equals Chapati price x 2	Cart updated correctly at each step; total recalculated instantly; removing item cleared it from list	**Passed**
TC03	Wallet order - sufficient funds	Top up wallet to KES 500 via staff,	Order document created in Firestore with status "pending";	Order appeared in Orders tab with status chip "Pending" and	**Passed**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
		add items totalling KES 150, select Wallet, tap Place Order	walletBalance decremented by 150; order appears in Orders tab	estimated wait "~5 min"; wallet balance reduced correctly	
TC04	Wallet order - insufficient funds	Set wallet balance to KES 50, add items totalling KES 200, select Wallet, tap Place Order	App blocks order; shows insufficient funds warning; no Firestore document created; balance unchanged	Snackbar appeared: "Insufficient wallet balance"; no order created; balance remained at KES 50	**Passed**
TC05	Cash order	Add items to cart, select Cash, tap Place Order	Order document created with paymentMethod "cash" and status "pending"; no wallet deduction	Order appeared in Orders tab with status "Pending"; wallet balance unchanged; paymentMethod field confirmed	**Passed**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
				as "cash" in Firestore	
TC06	Staff marks Preparing	Log in as staff, open StaffOrders tab, tap "Start Preparing" on student order	Order status updates to "preparing" in Firestore; student order chip turns blue	Staff tapped button; student Orders tab refreshed within ~1 second; status chip turned blue ("Preparing")	**Passed**
TC07	Staff marks Ready and FCM	Staff taps "Mark Ready" on order	Order status updates to "ready"; student receives FCM push notification; chip turns green	Status updated; push notification arrived on student device in ~3 seconds; chip turned green ("Ready")	**Passed**
TC08	Pre-order scheduling	Student opens Pre-orders tab, taps FAB, sets meal type Lunch	Pre-order document created in Firestore with status "scheduled";	Pre-order card appeared immediately with correct meal type, time,	**Passed**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
		and a future date/time, taps Save	appears in Pre-orders list	and "Scheduled" badge	
TC09	Staff wallet top-up	Staff opens StaffWallet tab, enters student ID, enters amount KES 200, taps Top Up	walletBalance incremented by 200; walletTransaction document created with type "credit" and staffId	Student balance updated; transaction recorded with correct staffId; student Profile/Wallet screen reflected new balance	**Passed**

All nine functional test cases passed. The complete student order lifecycle - from registration through menu browsing, cart management, order placement, real-time status tracking, and FCM notification - executed without errors on a physical Android device. The pre-order scheduling and staff wallet top-up flows likewise completed correctly, with Firestore documents reflecting the expected state after each operation. The wallet payment path correctly enforced the balance check in TC04, preventing an order from being placed when funds were insufficient, while the cash path in TC05 bypassed wallet logic entirely as designed.

7.2.2 Security Testing

Security testing verified that the Firestore security rules correctly enforce role-based access boundaries across all user types. Testing was conducted using the Firebase Console Rules Playground, which allows simulated read and write operations to be authenticated with specific user tokens and roles without modifying live data. The three scenarios targeted the most critical

security boundaries: student wallet self-modification, unauthenticated collection access, and staff elevated read permissions.

Test Cases - Table 7.2: Security Testing

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC 10	Student cannot self-modify wallet	In Firebase Console Rules Playground, simulate a write to users/{studentUid} with walletBalance: 99999 using a student auth token	Write denied by Firestore rules	Rules Playground returned DENIED; the student rule only permits writing to the orders and preOrders collections, not to the users document	**Passed**
TC 11	Unauthenticated read denied	In Rules Playground, simulate a GET on the orders collection with no auth token	Read denied	Rules Playground returned DENIED; the isAuth() guard blocks all unauthenticated access across every collection	**Passed**
TC 12	Staff can read all orders	In Rules Playground, simulate a GET on the orders collection using a	Read allowed	Rules Playground returned ALLOWED; the isPrivileged() rul	**Passed**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
		staff auth token (role = "staff")		e function (which includes isStaff() and isAdmin()) grants read access to all orders	

All three security test cases passed. The Firestore security rules correctly enforce all role boundaries tested. Unauthenticated requests are rejected at the outermost isAuth() guard before any collection-level rule is evaluated. Students are confined to their own data partitions and cannot modify sensitive fields such as walletBalance, which can only be written by privileged roles (staff and admin) via the StaffWallet interface. Staff and admin roles receive the correct elevated permissions through the shared isPrivileged() helper function. This rule architecture ensures that wallet integrity relies entirely on server-controlled staff operations rather than client-side writes.

7.2.3 Performance Testing

Performance testing measured the response times of three critical operations: application cold launch, real-time order status propagation from staff to student, and order appearance on the staff screen after a student places an order. These benchmarks were recorded on a physical mid-range Android device over a Wi-Fi connection against the live Firestore backend. The target threshold for all operations was under 3 seconds, consistent with the responsiveness requirements defined during the design phase.

7.3: Performance Testing

Test Cases - Table

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC13	Menu load time	Kill app process, cold-launch app, measure time from open to Menu screen fully populated with items	Menu items visible within 3 seconds	Menu loaded with all 13 items in approximately 1.8 seconds on a physical Android device over Wi-Fi	**Passed**
TC14	Order status propagation - staff to student	Staff marks order Ready on one device; observe student device for chip colour change	Status update visible on student screen within 3 seconds	Student order chip updated in approximately 1 second; Firestore real-time listener reflected the change near-instantly	**Passed**
TC15	Order appears on staff screen	Student places order; observe StaffOrders	New order visible on staff screen	Order appeared on staff screen in approximately 1–2 seconds; Firestore	**Passed**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
		screen on staff device	within 3 seconds	snapshot listener triggered on the new document	

All three performance test cases passed and met their targets comfortably. The Firestore real-time snapshot listeners provided sub-2-second propagation for order status changes in both directions, eliminating the need for manual polling. The cold-launch menu load time of approximately 1.8 seconds falls well within the 3-second target, and repeat launches are faster still due to Firestore client-side caching after the initial fetch. These results reflect the efficiency of the listener-based architecture and confirm that the system delivers a responsive real-time experience suitable for a live cafeteria environment.

7.3.1 Developer Role Simulation

In Stage 1, the developer operated two physical Android devices simultaneously - one logged in as a student and one logged in as a staff member - to simulate the complete order lifecycle end-to-end. The student device was used to register a new account, browse the menu, assemble a cart, and place both a wallet order and a cash order. The staff device was used to observe the incoming orders on the StaffOrders screen, tap "Start Preparing" to change the order status, and tap "Mark Ready" to trigger the FCM notification.

The pre-order scheduling flow was tested for a future lunch slot using the bottom sheet dialog accessed from the Pre-orders tab FAB. The wallet top-up and manual deduction features were exercised from the StaffWallet tab on the staff device, with the resulting balance change confirmed on the student device's Profile/Wallet screen.

All operations completed without errors. Key observations recorded during this stage:

- Bottom navigation between tabs was smooth and preserved scroll position correctly.

- The order status chip colour progression (grey for Pending, blue for Preparing, green for Ready) was visually clear and updated in approximately 1 second after each staff action.
- The wallet balance update was immediately reflected in the Profile tab after a staff top-up, confirming that the real-time Firestore listener on WalletViewModel was functioning correctly.
- The FCM push notification arrived on the student device within approximately 3 seconds of the staff tapping "Mark Ready", consistent with the performance test result in TC14.
- The pre-order card appeared in the Pre-orders tab immediately after saving, with the correct meal type, time, and "Scheduled" status badge.

7.3.2 Peer Review Session

In Stage 2, three peers from USIU-Africa participated in a structured review session. Two peers played the student role and one peer played the staff role, each using the application independently for approximately 15 minutes without developer guidance. Participants were given a brief verbal description of the app's purpose and then asked to explore it freely while the developer observed and noted feedback.

No functional failures were observed during the peer session. The following feedback was collected:

- *"Adding items and seeing the total change in the cart feels fast and natural."* - Student participant
- *"I wasn't sure what 'Pre-order' meant at first - a short tooltip or label would help."* - Student participant
- *"The notification when the order was ready was a nice touch - I would actually use this app."* - Student participant
- *"The wallet balance being visible in the cart before paying is very useful - you know immediately whether you have enough."* - Student participant

The staff participant navigated the StaffOrders screen without assistance and correctly identified the "Start Preparing" and "Mark Ready" buttons. The StaffWallet top-up flow was completed successfully on the first attempt.

The main usability observation from this stage was that the "Pre-orders" section label was not immediately self-explanatory to first-time users unfamiliar with the concept of advance scheduling. This is noted as a future enhancement - a short onboarding label or tooltip on first launch would improve discoverability - and is recorded as a minor UI refinement rather than a functional defect. Overall, the application was received positively, with participants expressing that the ordering flow was intuitive and that the real-time notification feature added significant value.

7.4 Key Findings

The evaluation of the USIU Cafeteria Ordering System confirmed that the application successfully meets its core functional, security, performance, and usability requirements. The testing process revealed both the strengths of the chosen architecture and a small number of areas identified for future improvement.

a) Functionality

All nine functional test cases passed. The core order lifecycle - menu browsing, cart management, wallet and cash payment, real-time status tracking, and FCM push notification - executed reliably end-to-end on a physical Android device. The pre-order cut-off automation, executed by the Supabase Edge Function triggered by cron-job.org at 10:00 AM for lunch and 5:00 PM for dinner, correctly deducted wallet balances and dispatched FCM alerts at the scheduled times. Wallet operations - top-up, deduction, and atomic order payment via Firestore runTransaction - maintained balance integrity across all tested scenarios with no partial writes or inconsistencies observed.

b) Performance

All three performance targets were met within the 3-second threshold. Menu cold-launch load time was approximately 1.8 seconds. Real-time order status propagation between staff and student devices consistently completed in approximately 1 second, enabled by Firestore snapshot listeners

that react to document changes without polling. Client-side Firestore caching further reduced repeat menu load times to near-instant after the first fetch. These results confirm that the system delivers the responsiveness expected of a live ordering environment.

c) Security

All three Firestore security rule tests passed. The role-based access model correctly blocked unauthenticated reads, prevented students from self-modifying the walletBalance field, and permitted staff to read all orders across all students. The isPrivileged() helper function cleanly combined the staff and admin roles, while the isAuth() outermost guard rejected all unauthenticated access before any collection-specific rule was evaluated. The admin web panel's API routes use the Firebase Admin SDK server-side, which operates outside client-facing rules and is inherently trusted.

d) Usability

The Material Design 3 component set and USIU branding (navy #002147 and gold #CFB991) produced a consistent and professional interface. Tab-based navigation was intuitive for both student and staff roles, and the cart quantity stepper, status chip colours, and real-time balance display were appreciated by UAT participants. The main usability finding from the peer review was that the "Pre-orders" section label was not immediately self-explanatory to first-time users. A short onboarding tooltip or contextual label would improve discoverability. This is noted as a minor future enhancement and does not constitute a functional defect.

7.5 Impact Analysis

The implementation of the USIU Cafeteria Ordering System has delivered measurable improvements in ordering efficiency, communication between students and staff, and cafeteria resource management. Its impact spans all three user roles and demonstrates the practical value of cloud-based mobile technology in a university setting.

i. Students

Students can browse the cafeteria menu, add items to a cart, and place orders from anywhere on campus without physically queuing at the counter. The in-app wallet removes the friction of carrying exact cash for every transaction. The real-time status chips and FCM push notification

inform students exactly when their meal is ready, eliminating the need to wait at the counter or repeatedly check back. The pre-order feature allows students with tight lecture timetables to schedule meals in advance and collect them at a guaranteed time, directly addressing the peak-hour queue problem identified in the problem statement.

ii. Cafeteria Staff

Cafeteria staff receive a real-time, structured view of all incoming orders on the StaffOrders screen, replacing the current ad-hoc verbal and handwritten counter system. The "Start Preparing" and "Mark Ready" actions communicate order status to students automatically, reducing verbal overhead at the counter. The StaffWallet interface enables staff to credit student accounts immediately upon receiving cash, with an audit trail that records the staff member's ID on every transaction - improving financial accountability without requiring a separate cashiering system.

iii. Administrators

The Next.js admin web panel gives administrators full menu and user management control without requiring Android device access. Administrators can add, edit, and toggle the availability of menu items in real time, and can create new staff accounts with a forced first-login password change for security. The centralised dashboard provides a directory of all registered students and staff, supporting oversight without direct database access.

iv. Platform Maintainability

The architecture separates concerns clearly across three tiers: the Android app handles student and staff interactions, the Next.js panel handles administration, and Supabase Edge Functions handle scheduled server-side logic. All three tiers connect to the same Firebase Firestore backend. This separation allows each component to be updated independently. Firestore's schemaless document model allows new fields to be added without schema migrations, as demonstrated during development by the addition of firstLogin, staffId, and fcmToken fields. The free-tier infrastructure - Firebase Spark, Supabase free tier, and Vercel - ensures zero operational cost for the prototype phase.

v. SDGs

SDG 4 - Quality Education: By reducing time lost to cafeteria queuing during short breaks between lectures, the system helps students spend more of their limited break time resting or preparing for class. Pre-ordering further supports students with tight timetables who would otherwise skip meals to avoid missing the start of a lecture.

SDG 9 - Industry, Innovation and Infrastructure: The system demonstrates that low-cost, cloud-based mobile solutions - built entirely on free tiers (Firebase Spark, Supabase, Vercel, cron-job.org) - can deliver production-grade university infrastructure without capital expenditure. This model is directly applicable to other Kenyan higher education institutions seeking to digitise manual campus processes.

SDG 11 - Sustainable Cities and Communities: Digitising the food ordering process reduces paper-based operations such as manual receipts and chalkboard menus. The pre-order visibility gives cafeteria staff advance demand data, enabling more accurate food preparation and reducing over-production and food waste - contributing to a more resource-efficient campus community.

7.6 Conclusion

The testing and evaluation phase confirmed that the USIU Cafeteria Ordering System is functionally complete, secure, and performant. All fifteen test cases - nine functional, three security, and three performance - passed against their defined criteria on a physical Android device running against the live Firebase backend. The complete student order lifecycle, the staff order management workflow, the in-app wallet with atomic Firestore transactions, the pre-order cut-off automation via Supabase Edge Functions, and the FCM push notification pipeline all operated as specified without critical defects. UAT validated that the application is usable by real participants without prior training, with feedback confirming that the ordering flow is intuitive and that real-time notifications add meaningful value. The only usability observation - the discoverability of the Pre-orders section for first-time users - has been noted as a minor future enhancement.

The impact analysis further demonstrated that the system addresses the core problem of cafeteria queue management while delivering broader benefits to staff accountability, administrator control, and campus sustainability. The free-tier cloud infrastructure model proved viable for a university prototype and provides a scalable foundation for future enhancements, including indoor navigation

integration, repeat-order shortcuts, and full admin analytics. The system is ready for deployment and provides a solid basis for the recommendations presented in Chapter 8.

8 CHAPTER 8: CONCLUSION

8.1 Introduction

This final chapter provides a comprehensive summary of the USIU Cafeteria Ordering System project, reflecting on its primary objectives, architectural decisions, implementation outcomes, and the technical challenges encountered during development. The application was built to address the persistent inefficiencies of the current manual cafeteria ordering process at USIU-Africa - long queues during peak hours, cash-only transactions, no advance ordering capability, and no real-time communication between students and cafeteria staff. Background research during the requirements phase confirmed that no existing campus-specific tool addressed this problem holistically. Existing general-purpose food delivery applications did not integrate with the campus wallet system, could not be operated by cafeteria staff without smartphones, and lacked the institutional identity required for adoption within a university environment. These insights shaped the vision for the USIU Cafeteria Ordering System: a purpose-built, three-role Android application backed by a real-time cloud database, designed to digitise the entire cafeteria workflow from menu display to meal collection notification.

The system's primary aims were to eliminate the physical queue for common orders, provide students with an in-app digital wallet, enable advance pre-ordering around lecture timetables, and give cafeteria staff a structured, real-time order management interface. Secondary objectives included building an admin web panel for menu and user management, enforcing role-based data security through Firestore rules, and deploying all server-side logic on free-tier infrastructure to demonstrate viability without capital expenditure.Development was executed in structured phases: requirements gathering, system design (UML, wireframes, class diagrams, data model), Android implementation across twelve incremental steps, Supabase Edge Function deployment, Firestore rules and index configuration, Next.js admin panel development, and rigorous manual testing. The iterative approach allowed issues such as the Deno gRPC incompatibility and the Firestore transaction atomicity design to be identified and resolved before they could affect the deployed system. Testing confirmed that all fifteen test cases - functional, security, and performance - passed on a physical Android device against the live backend. User Acceptance Testing with peer participants validated that the interface was intuitive across all three roles. The system is deployed

and operational, providing a working prototype that demonstrates the full end-to-end cafeteria ordering workflow and a clear technical foundation for future production deployment.

8.2 Achievements

Significant milestones were accomplished during the development of the USIU Cafeteria Ordering System, spanning all three application tiers and covering the full scope of the original requirements. Complete three-role system across two applications. The most substantial achievement was delivering a fully functional system serving three distinct user roles. Students and cafeteria staff use a native Android app - the student interface provides five tabs (Menu, Cart, Orders, Pre-orders, Profile/Wallet) while the staff interface provides three tabs (StaffOrders, StaffMenu, StaffWallet). Administrators use a separate Next.js web panel for menu and user management. Role-based routing from LoginActivity ensures that each Android user lands on the correct interface after authentication, with no manual configuration required.

Real-time order lifecycle with FCM notifications. The end-to-end order flow - from a student placing an order to receiving a push notification when the meal is ready - was implemented and verified on physical devices. Firestore snapshot listeners provide sub-second status propagation between the student and staff interfaces, and Firebase Cloud Messaging delivers the "order ready" notification reliably within approximately 3 seconds. This replaces all verbal communication between student and counter staff for routine orders. Atomic in-app wallet with full audit trail. The Firestore runTransaction implementation for wallet order placement atomically decrements the student balance, creates the order document, and records the transaction in a single operation with no risk of partial writes. The staff wallet top-up and deduction operations are equally atomic, and every transaction records the staffId of the staff member who performed it, providing a complete financial audit trail without a separate accounting system.

Pre-order scheduling with automated cut-off processing. The pre-order system - allowing students to schedule wallet-funded meals in advance with a 10:00 AM (lunch) or 5:00 PM (dinner) cut-off - was fully implemented and deployed. The Supabase Edge Function process-cutoff, triggered by cron-job.org at the scheduled times, queries all pending pre-orders, processes wallet deductions atomically, dispatches FCM confirmations or cancellation notices, and schedules recurring pre-

orders for the next occurrence. This server-side automation required no persistent server and runs entirely on the free tier.

Firestore security rules and composite indexes. Five composite Firestore indexes and a comprehensive set of security rules with role-based helper functions (isAuth(), isOwner(), isStaff(), isAdmin(), isPrivileged()) were deployed to production. The rules correctly enforce data partitioning - students can only read their own orders, staff can read all orders, and the walletBalance field is write-protected against client-side modification. All three security test cases confirmed these rules operate as specified. Next.js admin web panel deployed on Vercel. A separate Next.js 14 web application provides administrators with a dashboard for menu management, staff account creation (via the Firebase Admin SDK), and a directory of all registered users. The panel is deployed on Vercel's free tier with HTTPS, uses server-side Firebase Admin SDK calls exclusively, and requires no Android device access for administrative operations. Free-tier cloud infrastructure. The entire system - Firebase Firestore, Firebase Auth, FCM, Supabase Edge Functions, cron-job.org scheduling, and Vercel hosting - operates within free service tiers, incurring zero monthly cost. This demonstrates that production-grade, multi-tier university software can be built and deployed without infrastructure budget, which is a meaningful outcome for resource-constrained institutions.

8.3 Challenges Encountered

Although the USIU Cafeteria Ordering System was successfully built and deployed, the development process presented several technical and architectural challenges that required significant problem-solving effort. Firebase Admin SDK incompatibility in Deno edge runtime. The most significant technical challenge arose when implementing the Supabase Edge Functions. The Firebase Admin SDK uses gRPC internally for Firestore communication, but the Deno runtime used by Supabase Edge Functions does not support gRPC. Initial attempts to connect from the edge function to Firestore produced persistent connection errors that were not immediately obvious from the error messages. The resolution required explicitly configuring the Firebase Admin SDK with db.settings({ preferRest: true }) to force the SDK to use the REST API instead of gRPC. This single configuration line was critical and is not prominently documented; identifying it required deep investigation of the Firebase Admin SDK internals and Deno runtime limitations. Firestore transaction design for atomic wallet operations. Designing the wallet order

placement flow to be fully atomic required careful structuring of the Firestore runTransaction call. An early implementation performed the balance check and balance deduction as separate Firestore writes, which created a race condition where two simultaneous order placements could both pass the balance check before either had decremented the balance. Migrating the logic into a single runTransaction block - which reads the current balance, verifies sufficiency, decrements it, and creates the order document in one atomic operation - resolved the race condition but required significant restructuring of the FirestoreRepository and CartFragment interaction.

Fragment navigation and state preservation. An early architectural decision to use replace fragment transactions for the bottom navigation caused the CartViewModel to reset whenever the user navigated away from the Cart tab. This meant items added to the cart were lost if the user browsed back to the Menu. Migrating from replace to a hide/show fragment management pattern in MainActivity and StaffMainActivity preserved each fragment's lifecycle and ViewModel state across tab switches, resolving the issue, but required a full refactor of the navigation setup.First-login forced password change for staff accounts. Staff accounts are created by the admin via the Firebase Admin SDK with a temporary password. Ensuring that staff were forced to change this password on their first login required a firstLogin boolean field on the user document in Firestore and a ChangePasswordActivity that intercepted navigation immediately after login when the flag was true. Coordinating the flag reset (written after a successful updatePassword call) with the subsequent navigation to StaffMainActivity required careful sequencing to avoid race conditions between the Firestore write and the activity transition.

FCM token management. FCM tokens are generated per device and can be rotated by the platform. An early implementation stored the FCM token only at registration time, which caused notifications to stop arriving after a token refresh. The solution required implementing onNewToken in MyFirebaseMessagingService to write the refreshed token back to the user's Firestore document whenever FCM issued a new one. This ensured the fcmToken field on the users document was always current, regardless of when the rotation occurred.

Multi-role security rule design. Writing Firestore security rules that correctly handled three roles (student, staff, admin) across five collections without duplicating logic required several iterations. The initial rules were written with inline role checks per collection, leading to repetitive and error-

prone conditions. Refactoring to use shared helper functions (isAuth(), isOwner(), isStaff(), isAdmin(), isPrivileged()) and testing all boundary conditions via the Firebase Console Rules Playground produced a clean, maintainable rule set that passed all three security test cases.

8.4 Future Work

While the current version of the USIU Cafeteria Ordering System meets its core objectives and is deployed as a functional prototype, several enhancements are identified for future iterations that would improve usability, extend functionality, and prepare the system for full production scale. Online payment integration. The current wallet system requires students to deposit cash at the counter to credit their in-app balance. Integrating a mobile money gateway such as M-Pesa STK Push would allow students to top up their wallets remotely from anywhere, removing the last physical dependency on the counter for wallet management. This would significantly improve convenience for students and reduce the cash-handling workload for staff.

Pre-order discoverability and onboarding. UAT feedback identified that the "Pre-orders" section label was not immediately self-explanatory to first-time users. A future enhancement would add a brief onboarding screen or contextual tooltip on first launch, explaining how pre-ordering works and when cut-off times apply. This change requires minimal engineering effort but would meaningfully reduce the learning curve for new users. Push notification preferences. Currently, all FCM notifications (order ready, pre-order confirmation, pre-order cancellation) are sent without user configuration. A future notification preferences screen in the Profile/Wallet tab would allow students to opt out of specific notification types or set quiet hours, improving the experience for students who prefer not to receive notifications during lectures.

Order history analytics for staff and admin. The current system stores all orders in Firestore but provides no aggregate reporting. A future admin dashboard module - built into the existing Next.js admin panel - could display daily order volumes, peak hours, most popular menu items, and wallet transaction summaries. This data would help cafeteria management plan staffing levels and stock quantities more accurately, directly improving operational efficiency. Repeat order and favourites shortcut. A common friction point for regular users is rebuilding the same cart from scratch each

day. A future "Reorder" button on a completed order in the Orders history would populate the cart with the same items in one tap, significantly reducing the time required to place routine orders.

Cash payment confirmation by staff. Currently, cash orders appear on the StaffOrders screen identically to wallet orders, with no mechanism for staff to confirm that cash was physically received before preparing the order. A future enhancement would add a "Confirm Payment" step for cash orders, preventing kitchen preparation from beginning before payment is collected at the counter. Scalability to multiple cafeteria outlets. The current data model assumes a single cafeteria. Introducing a outletId field on menuItems and orders documents would allow the system to scale to multiple on-campus food outlets - such as a coffee kiosk or a snack bar - each with their own staff interface and menu, all under the same Firebase project and admin panel. Full automated test suite. The current testing approach relies on manual execution of test cases. A future iteration would introduce a JUnit and Mockito unit test suite for CartViewModel, WalletViewModel, and FirestoreRepository, along with Espresso UI tests for the critical order placement and wallet deduction flows. Automated testing would enable continuous integration checks on every code change and reduce regression risk as the feature set grows.

9 REFERENCES

Abdul, A. M., Mohammad, A. A. K., Reddy, P. V., Nuthakki, P., Kancharla, R., Joshi, R., & Raja, N. K. (2022). Enhancing security of mobile cloud computing by trust- and role-based access control. *Scientific Programming, 2022*, Article 9995023. https://doi.org/10.1155/2022/9995023

Android Developers. (2024). *Android developer guides*. Google. https://developer.android.com/guide

Dennis, A., Wixom, B. H., & Roth, R. M. (2020). *Systems analysis and design* (7th ed.). Wiley.

Epiloksa, H. A., Kusumo, D. S., & Adrian, M. (2022). Effect of MVVM architecture pattern on Android based application performance. *Jurnal Media Informatika Budidarma*, *6*(4). https://doi.org/10.30865/mib.v6i4.4545

Firebase. (2024). *Firebase documentation*. Google. https://firebase.google.com/docs

Google. (2024). *Material Design 3*. Google. https://m3.material.io/

Maniratnam, U., Pandey, S. K., Srivastava, U., Kumar, P., & Sharma, P. (2020). Automated canteen ordering system using Android. *International Journal for Research in Applied Science and Engineering Technology*, *8*(V). https://doi.org/10.22214/ijraset.2020.5154

Mokar, M. A., Fageeri, S. O., & Fattoh, S. E. (2019). Using Firebase Cloud Messaging to control mobile applications. In *Proceedings of the 2019 International Conference on Computer, Control, Electrical, and Electronics Engineering (ICCCEEE)* (pp. 1–5). IEEE. https://doi.org/10.1109/ICCCEEE46830.2019.9071008

Moroney, L. (2017). *The definitive guide to Firebase: Build Android apps on Google's mobile platform*. Apress. https://doi.org/10.1007/978-1-4842-2943-9

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education Limited.

Supabase. (2024). *Supabase documentation*. Supabase Inc. https://supabase.com/docs

10 APPENDIX A: USER MANUAL FOR THE USIU CAFETERIA ORDERING SYSTEM

This appendix serves as a quick-start guide for the three user roles of the USIU Cafeteria Ordering System: Students, Cafeteria Staff, and System Administrators.

A.1 For Students (Android App)

Getting Started

Registration: Open the app and tap **Register**. Enter your name, student ID, USIU email address, and create a password. Your in-app wallet will be created automatically with a balance of KES 0.00.

Login: Enter your email and password. The system will read your role from Firestore and direct you automatically to the Student Dashboard.

Placing a Regular Order

Browse the Menu: On the **Menu** tab, use the category chips at the top of the screen - All, Mains, Snacks, Drinks, Specials - to filter items. Items that are sold out display a red "Sold Out" chip and a disabled Add to Cart button. Tap **Add to Cart** on any available item. A snackbar will confirm the addition.

Review the Cart: Navigate to the **Cart** tab. Use the + and − stepper buttons to adjust item quantities. Removing the last unit of an item removes it from the cart entirely. The subtotal and an estimated wait time update in real time at the bottom of the list.

Select a Payment Method:

- **Wallet:** Select the Wallet option. Your current balance is shown alongside the toggle. If the balance is insufficient, a warning appears and the Place Order button is disabled. Top up your wallet at the cafeteria counter before proceeding.

- **Cash:** Select the Cash option. No balance check is performed. You will pay physically at the counter when you collect your order.

Tap **Place Order** to confirm. For wallet orders, the deduction and order creation happen atomically - if either fails, neither is committed.

Track Your Order: Go to the **Orders** tab. Your active order displays a three-step status bar that updates in real time without any manual refresh:

- *Pending* (grey chip) - order received, awaiting preparation.
- *Preparing* (blue chip) - cafeteria staff have started your meal.
- *Ready* (green chip) - your meal is ready for collection at the counter.

A push notification will also arrive on your device when the status changes to Ready, even if the app is in the background.

Smart Pre-order (Scheduling a Meal in Advance)

Navigate to the **Pre-orders** tab and tap the + button (bottom-right corner).

In the dialog that appears:

1. Select the **Meal Type**: Lunch or Dinner.
2. Choose a **Date** and **Pickup Time** for the scheduled meal.
3. Select your items and quantities from the menu list.
4. Optionally, toggle **Recurring** and select the day(s) of the week to repeat the pre-order automatically each week.

Tap **Save**. The pre-order will appear in your list with a "Scheduled" badge.

Important: Pre-orders are funded entirely from your wallet. No deduction occurs at scheduling time. The deduction is processed automatically at the cut-off time - 10:00 AM EAT for Lunch slots, 5:00 PM EAT for Dinner slots. Ensure your wallet has sufficient funds before the cut-off.

If the balance is insufficient at cut-off time, the pre-order will be cancelled and you will receive a push notification. You may then top up and reschedule.
To cancel a pre-order, tap the **Cancel** button on its card. Cancellation is only possible while the status is "Scheduled" - once the cut-off has processed and the status is "Confirmed", the order cannot be cancelled.

Managing Your Wallet

Go to the **Profile/Wallet** tab.

- **Balance:** Your current wallet balance is displayed in a prominent card at the top of the screen.
- **Top-up:** Wallet top-ups are handled at the cafeteria counter only. Bring cash and provide your Student ID to the staff member, who will credit your account. The updated balance will appear on your Profile screen within seconds.
- **Transaction History:** Scroll down below the balance card to see a full list of credits (top-ups) and debits (order payments and pre-order deductions), each with a date, description, and amount.
- **Logout:** Tap the Logout button at the bottom of the screen to sign out and return to the Login screen.

A.2 For Cafeteria Staff (Android App)

Login and First-Time Setup

Your account is created by the System Administrator. You will receive a temporary password via email or in person. Open the app, enter your email and temporary password, and tap **Login**. On your first login, you will be redirected automatically to a **Change Password** screen. You must set a new password before any other screen is accessible. This step is mandatory and cannot be skipped.

Processing Orders (StaffOrders Tab)

The **StaffOrders** tab is your primary operational screen. It displays all active orders from all students - those in Pending, Preparing, or Ready status - in a single real-time list. Each card shows the student's name, the ordered items and quantities, the total amount, the payment method (wallet or cash), and the current status.

Advance each order through the workflow using the action button on its card:

Action	Result
Tap **Start Preparing**	Status changes from Pending to Preparing. Student's in-app chip turns blue.
Tap **Mark Ready**	Status changes from Preparing to Ready. A push notification is sent automatically to the student's device. Student's chip turns green.
Tap **Mark Collected**	Order is finalised. For cash orders, collect physical payment from the student before tapping this button.

Orders disappear from the active list once marked Collected.

Managing the Menu (StaffMenu Tab)

- **Toggle Availability:** Flip the switch on any item card to instantly mark it as "Sold Out" or "Available". The change is reflected on all connected student devices within seconds.
- **Add Item:** Tap the + FAB. Fill in the Name, Description, Category, Price, and Image URL. Tap Save.
- **Edit or Delete:** Tap an existing item card to open the edit dialog. Modify any field and save, or tap Delete to remove the item permanently.

Managing Student Wallets (StaffWallet Tab)

This screen is used to credit or debit a student's in-app wallet balance in exchange for cash transactions at the counter.

1. Enter the student's **ID number** in the lookup field and tap **Find Student**. The student's name and current balance are displayed for confirmation.
2. Enter the **amount** of the transaction.
3. Tap **Credit** to add funds (top-up after receiving cash), or **Debit** to subtract funds (for corrections such as refunding an overcharge).
4. Tap **Confirm**. The transaction is recorded atomically and the student's balance updates immediately. Your staff ID is recorded on every transaction as part of the audit trail.

A.3 For Administrators (Web Panel)

Access

The admin panel is browser-based. Open a desktop browser and navigate to the deployed Vercel URL for your institution's admin panel. Log in with your administrator credentials. If this is your first login, you will be redirected to a password change page before accessing the dashboard.

Note: The admin panel is role-gated. Any user who does not hold the admin role will be redirected to the login page immediately after authentication.

Menu Management

The **Menu Management** page displays a full data table of all menu items. Each row shows the item name, category, price, current availability status, and image URL.

- **Add Item:** Click the **Add Item** button above the table. Fill in the Name, Description, Category (dropdown), Price, Image URL, and initial availability. Click Save. The item is immediately visible on all student Menu screens.
- **Edit:** Click the Edit (pencil) icon on any row. Modify the fields and save.
- **Delete:** Click the Delete (trash) icon on any row. This action is permanent.
- **Toggle Availability:** Flip the availability switch in any row to show or hide an item on the student menu without deleting it.

Staff Account Creation

Navigate to the **Staff Accounts** page.

Fill in the new staff member's **Full Name**, **Email**, and a **Temporary Password**. Click **Create Staff Account**. The system calls a server-side API route that:

1. Creates a Firebase Authentication account with the provided credentials.
2. Writes a Firestore user document with role: "staff" and firstLogin: true.

On their first login, the staff member will be forced to change the temporary password before accessing any operational screen. A success message on this page confirms the new account's email address.

Users Directory

The **Users Directory** page displays two separate tables:

- **Students:** Name, Student ID, Email, Wallet Balance, Date Joined.
- **Staff and Admins:** Name, Email, Role, First-Login Status (indicates whether the staff member has completed their first-login password change).

Both tables are read in real time from Firestore and reflect the current state of all registered accounts.

11 APPENDIX B: TECHNICAL DEPLOYMENT AND MAINTENANCE GUIDE

This appendix provides a technical reference for deploying, configuring, and maintaining the cloud infrastructure of the USIU Cafeteria Ordering System. It assumes familiarity with Firebase, Supabase, and Git.

B.1 Deployment Prerequisites

The following accounts and tools must be in place before deployment begins. All services operate on free tiers - no credit card or billing account is required.

Accounts required:

- Firebase (Spark free tier) - Firestore, Authentication, Cloud Messaging
- Supabase (free tier) - Edge Functions hosting
- Vercel (free tier) - Next.js admin panel hosting
- cron-job.org (free tier) - scheduled HTTP trigger jobs
- GitHub - source code repository

Tools required:

- Android Studio (Ladybug 2024.2.1 or newer) - for building and signing the Android APK
- Node.js v18 or newer - for Next.js admin panel development
- Git - version control
- Supabase CLI - for deploying and managing Edge Functions
- Firebase CLI - for deploying Firestore security rules and composite indexes

B.2 Environment Variables and Secrets

Sensitive credentials are never stored in source code. Each platform has its own secure storage mechanism. The table below lists every secret used by the system and where it is stored.

Platform	**Variable Name**	**Purpose**	**Where to Store**
Android App	FUNCTIONS_SECRET	Shared secret used to authenticate HTTP calls from the app to the	gradle.properties - excluded from version control via .gitignore. Exposed to app code at build time as BuildConfig.FUNCTIONS_SECRET.

Platform	Variable Name	Purpose	Where to Store
		Supabase Edge Functions.	
Supabase	FUNCTIONS_SECRET	Same shared secret - validated by Edge Functions on every incoming request via the Authorization: Bearer header.	supabase secrets set FUNCTIONS_SECRET=...
Supabase	FIREBASE_SERVICE_ACCOUNT	Firebase service account JSON key - used by the Firebase Admin SDK inside the Edge Functions to	supabase secrets set FIREBASE_SERVICE_ACCOUNT="$(cat key.json)"

Platform	Variable Name	Purpose	Where to Store
		read Firestore and send FCM notifications.	
Vercel	FIREBASE_ADMIN_*	Firebase Admin SDK service account credentials - used by Next.js server-side API routes to create staff accounts and write Firestore documents.	Vercel Project Settings → Environment Variables (server-side only, not exposed to the browser).
Vercel	NEXT_PUBLIC_FIREBASE_*	Client-side Firebase configuration (API key, project ID, etc.) - used by the	Vercel Project Settings → Environment Variables (prefixed NEXT_PUBLIC_, safe to expose to the client bundle).

Platform	Variable Name	Purpose	Where to Store
		Next.js browser client to authenticate and read Firestore within the bounds of security rules.	

B.3 Deployment Steps

Step 1 - Firebase Rules and Indexes

Deploy the Firestore security rules and composite indexes to the Firebase project using the Firebase CLI:

firebase deploy --only firestore:rules
firebase deploy --only firestore:indexes
Both commands read from firestore.rules and firestore.indexes.json in the project root. Run these commands once on initial setup and again whenever either file is modified.

Step 2 - Supabase Edge Functions

Navigate to the supabase/functions/ directory and deploy each function independently:
supabase functions deploy process-cutoff
supabase functions deploy notify-order-ready
Set the required secrets once using the Supabase CLI:

supabase secrets set FUNCTIONS_SECRET=your-shared-secret-here
supabase secrets set FIREBASE_SERVICE_ACCOUNT="$(cat service-account-key.json)"
Secrets are injected into the Deno runtime as environment variables at invocation time and are never stored in the function source code.

Critical implementation note: Both Edge Functions must include the line db.settings({ preferRest: true }) immediately after initialising the Firebase Admin SDK Firestore client. The Deno runtime used by Supabase does not support gRPC, which is the default transport protocol for the Firebase Admin SDK. Without this setting, all Firestore calls from the Edge Functions will fail silently. Setting preferRest: true forces the SDK to use the Firestore REST API, which is fully compatible with the Deno HTTP stack.

Step 3 - cron-job.org Scheduling

Create two HTTP POST jobs in cron-job.org with the following configuration:

Job Name	Endpoint URL	Schedule (UTC)	Request Body
cafeteria-lunch-cutoff	https://[project-ref].supabase.co/functions/v1/process-cutoff	0 7 * * * (07:00 UTC = 10:00 AM EAT)	{"mealSlot":"lunch"}
cafeteria-dinner-cutoff	https://[project-ref].supabase.co/functions/v1/process-cutoff	0 14 * * * (14:00 UTC =	{"mealSlot":"dinner"}

Job Name	Endpoint URL	Schedule (UTC)	Request Body
		5:00 PM EAT)	

For both jobs, add the following HTTP header to authenticate the request:

Authorization: Bearer YOUR_FUNCTIONS_SECRET
Replace [project-ref] with the Supabase project reference ID found in the Supabase project settings. Replace YOUR_FUNCTIONS_SECRET with the same value stored in the Supabase secrets.

Step 4 - Android APK Generation

1. In Android Studio, open the project and navigate to **Build → Generate Signed Bundle / APK**.
2. Select **APK** and click Next.
3. Select your release keystore (create one on first release using the **Create new** option - store the keystore file and passwords securely).
4. Select the **release** build variant and click Finish.
5. The signed APK is generated at app/build/outputs/apk/release/app-release.apk.

Install the APK directly on student and staff devices via USB or a file transfer mechanism. The app requires Android 7.0 (API Level 24) or higher.

Note: The FUNCTIONS_SECRET value must be present in gradle.properties before building. Confirm the line FUNCTIONS_SECRET=your-secret exists in that file and that gradle.properties is listed in .gitignore.

Step 5 - Next.js Admin Panel (Vercel)

1. Push the repository to GitHub (the admin panel source resides in the admin/ subdirectory).
2. In Vercel, click **Add New Project** and import the GitHub repository.
3. Set the **Root Directory** to admin in the Vercel project settings.
4. Add all environment variables listed in Section B.2 - both the server-side Admin SDK credentials and the NEXT_PUBLIC_ client-side Firebase config.
5. Click **Deploy**. Vercel will build and deploy the Next.js application automatically.

All subsequent pushes to the main branch on GitHub will trigger an automatic redeployment, making admin panel updates straightforward to roll out.

B.4 Firestore Composite Indexes

Firestore requires explicitly defined composite indexes for any query that combines a field filter with an ordering on a different field, or that applies multiple equality or range filters simultaneously. Without these indexes, the corresponding queries will throw a runtime error. The five indexes below must be deployed before the system is used.

Deploy via Firebase Console → Firestore Database → Indexes tab, or include them in firestore.indexes.json and deploy with firebase deploy --only firestore:indexes.

Collection	Index Fields	Query Purpose
orders	userId (Ascending) + createdAt (Descending)	Student order history - retrieve a student's past orders sorted by most recent.
orders	status (Ascending) + createdAt (Ascending)	Staff active order queue - retrieve all non-collected orders in arrival order.

Collection	Index Fields	Query Purpose
preOrders	userId (Ascending) + createdAt (Descending)	Student pre-order listing - retrieve a student's scheduled pre-orders.
preOrders	mealSlot (Ascending) + scheduledDate (Ascending) + status (Ascending)	Cut-off function batch query - retrieve all pre-orders for a given meal slot and date with status "scheduled".
walletTransactions	userId (Ascending) + createdAt (Descending)	Wallet transaction history - retrieve a student's wallet activity sorted by most recent.

B.5 Common Maintenance Tasks

The table below covers the most frequent operational and maintenance scenarios that system administrators and support staff will encounter.

Task	Procedure
Top up a student's wallet	Staff member uses the StaffWallet tab in the Android app: enter the student's ID, enter the amount, tap Credit. No admin panel action is needed.
Reset a staff member's password	The admin panel does not support password resets directly. Delete the staff account from the Users Directory page, then

Task	Procedure
	re-create it on the Staff Account Creation page with a new temporary password.
Remove a student account	Firebase Console → Authentication → locate the user → Delete. Also delete the corresponding document from the users collection in Firestore.
Add or update a menu item	Admin Web Panel → Menu Management → Add Item, or click Edit on an existing row. Changes are reflected on all student devices within seconds via Firestore snapshot listeners.
Check pre-order cut-off execution	Login to cron-job.org → select the relevant job → open the Execution History tab. Each successful run returns HTTP 200 with a JSON summary of confirmed and cancelled pre-orders.
Redeploy an Edge Function after a code change	Run supabase functions deploy process-cutoff or supabase functions deploy notify-order-ready from the project root. The updated function is live immediately.
Monitor Firestore quota usage	Firebase Console → Firestore Database → Usage tab. The Spark free tier allows 50,000 reads, 20,000 writes, and 20,000 deletes per day. Usage at prototype scale is well within these limits.
Rotate the FUNCTIONS_SECRET	Generate a new secret string. Update it in gradle.properties and rebuild the APK. Update it in Supabase using supabase secrets set

Task	Procedure
	FUNCTIONS_SECRET=new-value. Update it in both cron-job.org job headers. All four locations must be updated before distributing the new APK, otherwise Edge Function calls from the old APK will be rejected.
Add a new admin account	Firebase Console → Authentication → Add User. Then manually create a Firestore document in the users collection with the new UID and role: "admin". Set firstLogin: true to enforce a password change on first access to the admin panel.